MODULE 20 · DOMAIN 4 - MONITORING AND LOGGING

Health Checks and Self-Healing

Health at every layer: EC2 status checks and recovery, Auto Scaling and load balancer checks, containers, serverless failure handling, and designing checks that do not cause outages.

DOMAIN 4 of 6 EXAM WEIGHT 15% MODULE 20 of 34

admin@sthithapragna.com

Self-healing is a loop: something **checks**, something **decides**, something **acts**. And in the cloud, **replacing** usually beats repairing.

Health checks, layer by layer

LAYER	CHECK	NOTICES
Physical host	EC2 system status check	AWS hardware or network failure
Operating system	EC2 instance status check	Kernel panic, exhausted memory, bad config
Storage	Attached EBS status check	Volumes that cannot complete I/O
Application endpoint	Load balancer target health check	The app not answering correctly
Container	ECS health check, Kubernetes probes	A hung or unready container
Whole service	Route 53 health check, Synthetics	The service failing from outside

ON THE EXAM

A check only notices failures at its own layer. EC2 status checks pass happily while the application is down.

EC2 status checks, and what to do

CHECK FAILS	WHOSE PROBLEM	FIX
System status	AWS: the host	Recover, or stop and start to move host
Instance status	Yours: the OS	Reboot, then fix the cause
Attached EBS (August 2024, Nitro)	Storage path	Replace the volume or the instance

ON THE EXAM

Rebooting a system check failure stays on the same broken host. Recovery or stop and start moves it.

Automatic recovery for single instances

- **Simplified automatic recovery** is on by default for supported instance types.
- A CloudWatch alarm on **StatusCheckFailed_System** with the **recover** action does it explicitly.
- Recovery keeps the instance ID, private IPs, Elastic IP, EBS volumes and metadata.
- Not for instances with instance store volumes. For OS failures, use the reboot action instead.

> ON THE EXAM
>
> The recover action belongs on the system check. For instance check failures, the matching action is reboot.

Auto Scaling health check types

TYPE	ON BY DEFAULT?	REPLACES INSTANCES WHEN
EC2	Yes, always	System or instance status checks fail
ELB	No, turn it on	The load balancer marks the target unhealthy
EBS	No, turn it on	Attached EBS status check fails
VPC Lattice	No, turn it on	VPC Lattice marks the target unhealthy
Custom	Your code	SetInstanceHealth marks it unhealthy

ON THE EXAM

App crashed, load balancer says unhealthy, yet the group keeps the instance: ELB health checks were never turned on.

Load balancer target health

- Settings: path, interval, timeout, healthy and unhealthy thresholds, and success codes.
- Unhealthy targets stop receiving new requests, and return once healthy again.
- If **every** target is unhealthy, an ALB **fails open** and sends traffic to all of them.
- Fail open assumes the check is wrong, since everything failing at once is suspicious.

> ON THE EXAM
>
> Fail open protects traffic, but an Auto Scaling group using ELB checks may still replace every instance.

Shallow or deep health checks?

SHALLOW

Can this instance serve?

- Process running, local resources fine
- No calls to shared dependencies
- Safe for load balancers and replacement
- Fails only when this instance is broken

DEEP

Does the whole path work?

- Calls the database and downstream APIs
- Fails on every instance when a dependency fails
- Right for alarms, canaries, Route 53 failover
- Dangerous when it drives replacement

THE TIEBREAKER

A deep check that triggers replacement turns a brief database blip into every instance being terminated.

Container health

- An ECS task definition can include a **health check command**, with interval, retries and a start period.
- An ECS service replaces tasks whose essential containers become unhealthy.
- With a load balancer, both the target health check and the container check matter.
- On EKS: startup, readiness and liveness probes, as in module 7.

> ON THE EXAM
>
> Workers with no load balancer still need health: a container health check lets ECS replace hung tasks.

Serverless failure handling

- Asynchronous Lambda retries twice by default, within a maximum event age.
- **On-failure destinations** send the event plus error details to SQS, SNS, Lambda or EventBridge.
- Stream sources can split failing batches in two, limit retries, and send failures to a destination.
- SQS moves repeatedly failing messages to a **dead-letter queue**, and can redrive them back later.

> ON THE EXAM
>
> Serverless self-healing is retry, then set aside for later: nothing is lost, and nothing blocks the rest.

Dead-letter queue, or on-failure destination?

ON-FAILURE DESTINATION

Event plus context

- Includes the error and invocation details
- SQS, SNS, Lambda or EventBridge
- Also on-success destinations
- The newer, richer option

DEAD-LETTER QUEUE

The event alone

- Stores the original event payload
- SQS or SNS
- Failure only
- Long-established

THE TIEBREAKER

Keep failed events with the reason they failed: an on-failure destination.

Managed services heal themselves; clients must cooperate

- RDS Multi-AZ fails over automatically; Aurora promotes a replica; ElastiCache fails over its primary.
- Failover changes what a DNS endpoint points to. Clients must reconnect and not cache DNS for long.
- Retries with exponential backoff and jitter ride out brief failovers without a stampede.
- An application that never reconnects turns a 30-second failover into an outage.

ON THE EXAM

Database failed over but the application stayed broken: connection handling or DNS caching in the client.

AWS Health: failures announced in advance

- AWS Health reports events affecting your resources, including scheduled retirements and maintenance.
- Events arrive in EventBridge, so a rule can act before the deadline.
- For example: stop and start an instance scheduled for retirement, moving it to new hardware.
- An organisational view gathers Health events across all accounts.

ON THE EXAM

Handle scheduled instance retirements automatically: an EventBridge rule on AWS Health events. Module 21 goes deeper.

The self-healing loop, designed well

01
Check
At the right layer, shallow if it drives action.

02
Decide
Thresholds and grace periods avoid false positives.

03
Act
Replace, recover, restart, or fail over.

04
Limit
Replace a few at a time, not everything at once.

05
Tell
Emit an event so people know it happened.

ON THE EXAM

Self-healing that hides repeated failures is a problem too. Always record and alarm on how often it acts.

> Queue-processing workers run on Nitro-based EC2 instances in an Auto Scaling group, with no load balancer. Occasionally an attached EBS volume becomes impaired and the worker hangs.
>
> The instances' system and instance status checks still pass, so the group never replaces them.

What should the DevOps engineer do?

A Turn on Amazon EBS health checks for the Auto Scaling group

B Enable detailed monitoring on the instances

C Increase the health check grace period

D Add a CloudWatch alarm with the recover action to each instance

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A EBS health checks for the group

The group then acts on the attached EBS check, and replaces instances whose volumes are impaired.

B Detailed monitoring

More frequent metrics. Nothing acts on them.

C Longer grace period

Delays checking further. The opposite of what is needed.

D Recover alarm

For single instances with host failures. In a group, replacement is simpler, and this is a storage failure.

THE SIGNAL

The storage layer has its own check. Auto Scaling only acts on it once EBS health checks are turned on.

A web application runs on an Auto Scaling group behind an Application Load Balancer. Sometimes the application process crashes while the instance keeps running.

The load balancer marks those targets unhealthy and stops sending them traffic, but the group never replaces them, and capacity slowly drains away.

What should the DevOps engineer change?

A Lower the load balancer's unhealthy threshold

B Add a CloudWatch alarm with the recover action to each instance

C Add a launch lifecycle hook to the group

D Turn on ELB health checks for the Auto Scaling group, with a suitable grace period

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A **Lower threshold**

The load balancer already detects it. The group is not listening.

B **Recover alarm**

Watches the host. The host is fine; the process crashed.

C **Lifecycle hook**

Acts at launch or termination. Nothing is being terminated.

D **ELB health checks for the group**

The group then replaces instances the load balancer marks unhealthy.

THE SIGNAL

Load balancer knows, Auto Scaling does not act: ELB health checks are off for the group.

> A licensed application runs on a single EC2 instance, not in an Auto Scaling group. Occasionally the operating system runs out of memory and stops responding, and the instance status check fails.
>
> An existing CloudWatch alarm on StatusCheckFailed_Instance uses the recover action, but nothing happens when it fires.

What should the DevOps engineer change?

A Increase the alarm's evaluation periods

B Change the alarm's action to reboot the instance

C Move the instance into an Auto Scaling group of one

D Enable detailed monitoring

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A More evaluation periods

The alarm fires correctly. The action is the problem.

B Reboot action

Instance check failures are inside the OS; reboot is the matching action. Recover is for host failures.

C Group of one

Replacement brings a new instance ID and IP, which a licensed server cannot accept.

D Detailed monitoring

Faster data. The action still does not match the failure.

THE SIGNAL

Recover pairs with the system check; reboot pairs with the instance check.

> A web tier's load balancer health check calls /health, which queries the shared database. The Auto Scaling group uses ELB health checks.
>
> During a 90-second database failover, every instance failed its health check, and the group terminated and replaced the whole fleet, causing a 20-minute outage.

What should the DevOps engineer do?

A. Make the load balancer health check shallow, and monitor database health with separate alarms

B. Double the unhealthy threshold on the target group

C. Turn off ELB health checks for the Auto Scaling group

D. Add more instances to the Auto Scaling group

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A Shallow check, separate alarms

Instances fail only when they are broken; database problems alert people without killing the fleet.

B Higher threshold

Delays the same mass replacement. A longer failover still triggers it.

C ELB checks off

Stops the mass replacement, and also stops replacing genuinely crashed instances.

D More instances

More instances, all failing the same check, all replaced.

THE SIGNAL

Every instance failed at once: the check depends on something shared. Keep replacement checks shallow.

S3 uploads invoke a Lambda function asynchronously. When processing fails after all retries, the event is lost, and developers cannot tell why it failed.

The team wants every failed event kept together with the error that caused it, so it can be diagnosed and reprocessed, with the least effort.

What should the DevOps engineer configure?

A Increase the function's timeout and memory

B Enable detailed CloudWatch metrics for the function

C An on-failure destination that sends failed invocations to an SQS queue

D Invoke the function synchronously instead

◆ PAUSE AND DECIDE

What should the DevOps engineer configure?

A **More timeout and memory**

Might reduce some failures. Failed events are still lost, with no reason recorded.

B **Detailed metrics**

Counts failures. Keeps nothing.

C **On-failure destination**

Keeps each failed event with its error details, ready to diagnose and reprocess.

D **Synchronous invocation**

S3 notifications invoke asynchronously. The caller cannot be changed.

THE SIGNAL

Failed event plus the reason: on-failure destination. A dead-letter queue keeps the event alone.

> Background worker containers run as an ECS service with no load balancer. Sometimes a worker deadlocks: the container keeps running, but processes nothing.
>
> The team wants ECS to detect hung workers and replace them automatically. Each worker can report whether it has completed a job in the last two minutes.

What should the DevOps engineer do?

A. Put an Application Load Balancer in front of the workers

B. Alarm on the service's CPU utilisation

C. Enable the deployment circuit breaker

D. Add a container health check to the task definition that runs the worker's own check

◆ PAUSE AND DECIDE

What should the DevOps engineer do?

A **Add a load balancer**

Workers serve no requests. A load balancer adds cost to check the wrong thing.

B **CPU alarm**

A deadlocked worker may use little CPU, like an idle one. And it replaces nothing.

C **Circuit breaker**

Watches deployments, not tasks that hang long after starting.

D **Container health check**

Runs the worker's own check; ECS replaces tasks that become unhealthy.

THE SIGNAL

No load balancer, still needs health: a container health check. The best check is the application's own.

Module 20, and Domain 4, in five lines

- Each check sees only its own layer: system, instance, attached EBS, target, container, service.
- Recover for system checks; reboot for instance checks. Auto Scaling acts only on checks you turn on.
- Keep checks that drive replacement shallow; deep checks belong in alarms and failover.
- Serverless: retry, then on-failure destinations or dead-letter queues. Nothing lost.
- Clients must reconnect after managed failovers. AWS Health events can be acted on in advance.

ON THE EXAM

Domain 4 complete. Next, module 21 opens Domain 5: event-driven operations with EventBridge.